TP Audit Oracle 19c

Cas d'usage : le CHU Al Majd de Tanger



Mr. BADIR HASSAN
ENSA Tanger · 2026/2027

ÉNONCÉ

I MISE EN PLACE 4

R RAPPELS : BASE, SCHÉMA, VUE ET DONNÉES SENSIBLES 5

1 LE CONTEXTE 7

1.1 Le CHU Al Majd 7

1.2 Les acteurs 7

1.3 Les exigences de sécurité 8

1.4 Organisation du TP 9

2 LE JEU DE DONNÉES 10

2.1 Le modèle 10

2.2 Chargement 11

II CONTRÔLE D'ACCÈS 12

R RAPPELS : UTILISATEURS, PROFILS, PRIVILÈGES ET RÔLES

3 UTILISATEURS ET PROFILS 16

3.1 Quatre profils 16

3.2 Les comptes 17

3.3 Tester les profils 18

4 PRIVILÈGES ET RÔLES 19

4.1 La matrice RBAC 19

4.2 Créer les rôles 20

4.3 Vérifier, et voir les limites 20

III AUDIT 21

R RAPPELS : AUDIT STANDARD, AUDIT UNIFIÉ, FGA ET TRIGG

5 AUDIT STANDARD 26

5.1 Activer l'audit trail 26

5.2 Configurer l'audit 26

5.3 Lire la trace 27

6 AUDIT UNIFIÉ 28

6.1 Politiques du CHU 28

6.2 La DPO, auditrice 28

7 AUDIT FIN (FGA) 29

7.1 Signaler l'accès aux dossiers confidentiels 29

8 TRIGGERS D'AUDIT 30

8.1 Les journaux du responsable applicatif 30

8.2 Tracer les valeurs 30

8.3 Faire respecter une règle métier 30

8.4 Connexions et structure 31

IV ENQUÊTE 32

R RAPPELS : MÉTHODE D'ENQUÊTE ET EXPLOITATION DES TRAC

9 LES INCIDENTS DE LA NUIT 35

9.1 Le signalement 35

9.2 L'enquête 35

9.3 Le rapport 36

10 SYNTHÈSE 37

10.1 Questions de synthèse 37

10.2 Barème indicatif 37

V ANNEXES 38

11 AIDE-MÉMOIRE 39

11.1 Vues utiles 39

11.2 Codes d'erreur rencontrés 39

11.3 Fichiers fournis 40

MISE EN PLACE



Les données d'un hôpital : un patrimoine à protéger

R. RAPPELS

BASE, SCHÉMA, VUE ET DONNÉES SENSIBLES

R. DÉFINITIONS

CDB et PDB
Une base conteneur (CDB) héberge des bases enfichables (PDB). Chaque application vit dans une PDB ; le TP se déroule entièrement dans la PDB de travail (`ORCLPDB`).

Utilisateur et schéma
Un utilisateur est un compte ; son schéma est l'ensemble des objets qu'il possède (`chu.patients` = table `PATIENTS` du schéma `CHU`).

Compte schéma seul
Compte créé avec `NO AUTHENTICATION` : il possède des objets mais personne ne peut s'y connecter avec un mot de passe. Idéal pour le propriétaire des données d'une application.

Clé primaire, clé étrangère
La clé primaire identifie une ligne ; la clé étrangère (`REFERENCES`) garantit qu'une ligne pointe vers une ligne existante d'une autre table.

Vue
Requête nommée, stockée dans la base. En sécurité, la vue sert de « guichet » : on accorde la vue (quelques colonnes, quelques lignes) au lieu de la table.

SYS_CONTEXT('USERENV', ...)
Fonction qui renvoie des informations sur la session : `SESSION_USER`, `PROXY_USER`, `HOST`, `IP_ADDRESS`, `MODULE`, `CLIENT_PROGRAM_NAME`. Elle alimente les traces et les conditions d'audit.

Donnée de santé
Donnée personnelle sensible (loi 09-08 au Maroc, contrôlée par la CNDP), couverte par le secret médical : son accès doit être limité au besoin et traçable.

R. SYNTAXES

syntaxe

```
-- Se placer dans la PDB et exécuter un script
ALTER SESSION SET CONTAINER = orclpdb;
SHOW CON_NAME
@01_schema_dataset.sql

-- Compte propriétaire sans mot de passe
CREATE USER chu NO AUTHENTICATION
  DEFAULT TABLESPACE users QUOTA UNLIMITED ON users;

-- Table avec contraintes
CREATE TABLE chu.sejours (
  sejour_id  NUMBER PRIMARY KEY,
  patient_id NUMBER NOT NULL REFERENCES chu.patients,
  motif      VARCHAR2(60));

-- Vue de sécurité : seulement certaines colonnes et certaines lignes
CREATE VIEW chu.v_soins AS
  SELECT d.dossier_id, d.diagnostic, d.traitement
  FROM   chu.dossiers_medicaux d
  WHERE  d.confidentiel = 'N';

-- Informations de session
SELECT SYS_CONTEXT('USERENV','SESSION_USER'), SYS_CONTEXT('USERENV','HOST')
    FROM dual;
```

1. LE CONTEXTE

1.1. LE CHU AL MAJD

Le **CHU Al Majd de Tanger** (établissement fictif) vient de migrer son système d'information hospitalier sur une base **Oracle Database 19c**. Vous êtes recruté comme administrateur de base de données. La direction et la déléguée à la protection des données (DPO) vous confient une mission : rendre la base *sûre* et *auditable*.

> Les données de santé comptent parmi les plus sensibles qui soient. Au Maroc, la loi 09-08 et la CNDP encadrent leur traitement ; le secret médical s'impose à tout le personnel. Un incident (fuite, modification frauduleuse d'un dossier) engage la responsabilité de l'hôpital.

Le TP se déroule en trois temps : **protéger** la base avec les utilisateurs, les profils, les privilèges et les rôles ; **auditer** avec l'audit standard, l'audit unifié, l'audit fin et des triggers ; puis **enquêter** sur une série d'incidents provoqués par l'enseignant.

1.2. LES ACTEURS

Les acteurs du CHU et leurs comptes Oracle

Fonction	Comptes	Besoin métier
Médecin	`DR_ALAMI`, `DR_BENALI`, `DR_FASSI`	consulter et mettre à jour les dossiers médicaux ; prescrire
Chef de service	`DR_BENALI`	habiliter les infirmiers de son service (délégation)
Infirmier	`INF_GHAZI`	consulter les soins à réaliser (dossiers non confidentiels) et les prescriptions
Secrétaire	`SEC_KETTANI`, `SEC_LAHLOU`	accueillir : identité des patients, séjours ; aucune donnée médicale
Pharmacien	`PHA_MANSOURI`	consulter les prescriptions et marquer leur délivrance
Comptable	`CPT_OUAZZANI`	gérer les factures
Directeur	`DIR_RAMI`	statistiques d'activité, sans donnée nominative
DPO / auditrice	`DPO_SBAI`	lire les traces d'audit ; aucune donnée médicale
Application	`APP_CHU`	compte technique du logiciel hospitalier
Développeur	`DEV_CHU`	déployer dans le schéma `CHU` par proxy

1.3. LES EXIGENCES DE SÉCURITÉ

Exigences de la direction et de la DPO

- E1. Une personne, un compte : aucun compte partagé, aucun mot de passe applicatif connu des humains.
- E2. Moindre privilège : chaque fonction n'accède qu'aux données nécessaires (secret médical).
- E3. Les droits sont attribués par des **rôles** métier, jamais directement (sauf exception justifiée).
- E4. Mots de passe robustes, verrouillage après échecs, sessions limitées ; règles renforcées pour la direction et la DPO.
- E5. Toute consultation d'un dossier **confidentiel** (oncologie, patients VIP) est tracée et signalée.
- E6. Toute modification d'un dossier médical est tracée **avec l'ancienne et la nouvelle valeur**.
- E7. Un médecin ne modifie que ses propres prescriptions ; les tentatives sont tracées.
- E8. La gestion des comptes et des habilitations est tracée ; les échecs de connexion aussi.
- E9. La structure du schéma est protégée contre `DROP` et `TRUNCATE`.
- E10. La DPO lit les traces sans pouvoir les modifier ni consulter les données médicales.

Pourquoi l'audit est indispensable ici

Les exigences E1 à E4 relèvent du contrôle d'accès : utilisateurs, profils, privilèges, rôles. Mais un médecin autorisé peut consulter le dossier d'une personnalité hospitalisée dans un autre service : aucun privilège n'est violé. Seuls l'audit (E5 à E8) et les triggers (E6, E7, E9) permettent de *voir* et de *prouver* ce qui s'est passé.

1.4. ORGANISATION DU TP

Déroulement conseillé (durée totale : 8 h)

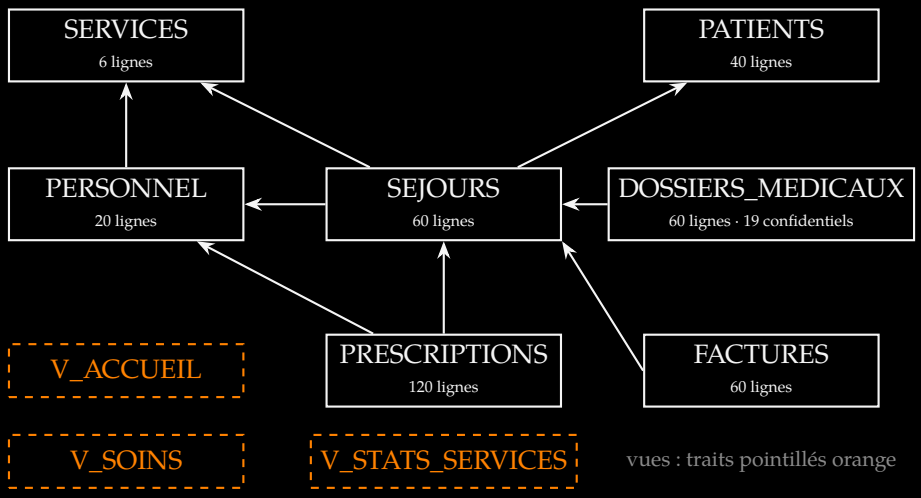
Ch.	Partie	Contenu	Durée
2	**Jeu de données**	schéma `CHU`, chargement, exploration	0 h 45
3	**Utilisateurs et profils**	4 profils, 12 comptes, proxy	1 h
4	**Privilèges et rôles**	matrice RBAC, 9 rôles, tests	1 h 15
5	**Audit standard**	`AUDIT_TRAIL`, `AUDIT`, `DBA_AUDIT_TRAIL`	0 h 45
6	**Audit unifié**	politiques, conditions, DPO	1 h
7	**Audit fin**	FGA sur les dossiers confidentiels	0 h 30
8	**Triggers d'audit**	journaux, contrôle, connexion, DDL	1 h 15
9--10	**Enquête et synthèse**	incidents, rapport, questions	1 h 30

Livrables : un script SQL commenté par chapitre, et un **rapport d'enquête** (chapitre 9). Toutes les commandes s'exécutent dans la PDB de travail (`ORCLPDB`, à adapter) ; on note **[SYS]** une commande d'administrateur et **[NOM]** une commande exécutée par le compte `NOM`.

2. LE JEU DE DONNÉES

2.1. LE MODÈLE

Le schéma `CHU` contient sept tables et trois vues. C'est un compte *schéma seul* (`NO AUTHENTICATION`) : personne ne s'y connecte directement.



Le modèle du schéma CHU

Les tables et les vues sensibles

Objet	Contenu et sensibilité
`PATIENTS`	identité, CIN, téléphone, mutuelle, indicateur `VIP`
`SEJOURS`	admission : patient, service, médecin, dates, motif
`DOSSIERS_MEDICAUX`	diagnostic, antécédents, traitement, indicateur `CONFIDENTIEL` (très sensible)
`PRESCRIPTIONS`	médicament, dose, médecin prescripteur, statut
`FACTURES`	montant, part de la mutuelle, statut
`V_ACCUEIL`	identité + séjour, sans donnée médicale (secrétariat)
`V_SOINS`	diagnostic et traitement des dossiers **non** confidentiels (infirmiers)
`V_STATS_SERVICES`	séjours, durée moyenne et chiffre d'affaires par service (direction)

2.2. CHARGEMENT

Le script `01_schema_dataset.sql`, fourni avec ce sujet, crée le schéma et insère toutes les données (fictives). En voici le début :

01_schema_dataset.sql — extrait

```
CREATE USER chu NO AUTHENTICATION DEFAULT TABLESPACE users QUOTA UNLIMITED ON
    users;

CREATE TABLE chu.dossiers_medicaux (
  dossier_id   NUMBER PRIMARY KEY,
  sejour_id    NUMBER NOT NULL UNIQUE REFERENCES chu.sejours,
  diagnostic   VARCHAR2(80),
  antecedents  VARCHAR2(80),
  traitement   VARCHAR2(80),
  confidentiel CHAR(1) DEFAULT 'N' CHECK (confidentiel IN ('O','N')),
  maj_le       DATE);

INSERT INTO chu.patients VALUES (7, 'K155433', 'Cherkaoui', 'Malak',
  DATE '1989-12-22', 'F', '0661008638', 'Tétouan', 'AUCUNE', 'O');
```

Exercice 2.1 Charger et contrôler le jeu de données

1. [SYS] Connectez-vous à la PDB et exécutez `@01_schema_dataset.sql`.
2. Comptez les lignes de chaque table. Comparez avec la figure : 6, 20, 40, 60, 60, 120, 60.
3. Vérifiez dans `DBA_USERS` que `CHU` a le type d'authentification `NONE`. Pourquoi ce choix pour un schéma applicatif ?

Exercice 2.2 Explorer les données sensibles

1. Combien de dossiers sont confidentiels ? Dans quels services ?
2. Quels patients sont VIP ? Dans quel service ont-ils séjourné et quel médecin les a suivis ?
3. Combien de séjours sont en cours (sans date de sortie) ?
4. Combien de lignes renvoient `V_SOINS` et `V_ACCUEIL` ? Expliquez la différence.

CONTRÔLE D'ACCÈS



Chacun sa rame : des droits adaptés à chaque fonction

R. RAPPELS

UTILISATEURS, PROFILS, PRIVILÈGES ET RÔLES

R. DÉFINITIONS

Authentification
Vérification de l'identité à la connexion : mot de passe (`IDENTIFIED BY`), système d'exploitation (`EXTERNALLY`), annuaire (`GLOBALLY`) ou aucune (`NO AUTHENTICATION`).

État d'un compte
`OPEN`, `LOCKED` (par le DBA), `LOCKED(TIMED)` (après trop d'échecs), `EXPIRED`, `EXPIRED(GRACE)` : visible dans `DBA_USERS.ACCOUNT_STATUS`.

Profil
Ensemble nommé de limites attribué à un compte : ressources (sessions, inactivité, durée) et mots de passe (échecs, verrouillage, durée de vie, réutilisation, complexité). Tout compte a un profil, `DEFAULT` par défaut.

Fonction de vérification
Fonction PL/SQL qui contrôle la complexité d'un mot de passe : `ORA12C_VERIFY_FUNCTION` (8 caractères, lettre et chiffre) ou `ORA12C_STRONG_VERIFY_FUNCTION` (9 caractères, 2 majuscules, 2 minuscules, 2 chiffres, 2 caractères spéciaux).

Privilège système
Droit d'effectuer une action dans la base (`CREATE SESSION`, `CREATE TABLE`, `SELECT ANY TABLE`...). Retransmissible avec `WITH ADMIN OPTION` ; la révocation n'est *pas* propagée.

Privilège objet
Droit sur un objet précis (`SELECT`, `INSERT`, `UPDATE`, `DELETE`, `EXECUTE`, `READ`) ; `INSERT`/`UPDATE` peuvent viser des colonnes. Retransmissible avec `WITH GRANT OPTION` ; la révocation est propagée en cascade.

Rôle et RBAC
Un rôle regroupe des privilèges et d'autres rôles. Le RBAC (*Role-Based Access Control*) accorde les privilèges aux rôles métier, et les rôles aux personnes.

Moindre privilège
Chaque compte ne reçoit que ce qui est strictement nécessaire à sa fonction.

SQL92_SECURITY
À `TRUE` (défaut depuis 12.2), un `UPDATE`/`DELETE` dont la clause `WHERE` lit des colonnes exige aussi le privilège `SELECT`.

Connexion proxy
Un utilisateur se connecte *à travers* un autre compte (`dev_chu[chu]`) : il agit

comme CHU, mais son identité réelle reste visible (PROXY_USER).

ORA-00942 / ORA-01031

00942 : aucun privilège sur l'objet (il est invisible). 01031 : l'objet est visible mais l'action n'est pas permise.

R. SYNTAXES : COMPTES ET PROFILS

syntaxe

```
CREATE USER nom IDENTIFIED BY "MotDePasse"
  DEFAULT TABLESPACE users TEMPORARY TABLESPACE temp
  [QUOTA 10M ON users] [PROFILE p] [PASSWORD EXPIRE] [ACCOUNT LOCK];

ALTER USER nom ACCOUNT LOCK | ACCOUNT UNLOCK | PASSWORD EXPIRE;
ALTER USER nom PROFILE p;
ALTER USER nom IDENTIFIED BY "Nouveau" REPLACE "Ancien";   -- par
    l'utilisateur lui-même
ALTER USER cible GRANT CONNECT THROUGH proxy;               -- connexion
    proxy[cible]
DROP USER nom CASCADE;

CREATE PROFILE p LIMIT
  SESSIONS_PER_USER 2        IDLE_TIME 15          CONNECT_TIME 480
  FAILED_LOGIN_ATTEMPTS 3    PASSWORD_LOCK_TIME 1/48            -- jours
    (ici 30 min)
  PASSWORD_LIFE_TIME 60      PASSWORD_GRACE_TIME 5
  PASSWORD_REUSE_TIME 365    PASSWORD_REUSE_MAX 10
  PASSWORD_VERIFY_FUNCTION ora12c_verify_function;
ALTER PROFILE p LIMIT IDLE_TIME 30;
DROP PROFILE p CASCADE;                                       -- les comptes
    repassent en DEFAULT
```

R. SYNTAXES : PRIVILÈGES ET RÔLES

syntaxe

```
GRANT CREATE SESSION, CREATE TABLE TO nom [WITH ADMIN OPTION];
GRANT SELECT, INSERT ON chu.patients TO nom [WITH GRANT OPTION];
GRANT UPDATE (telephone, ville) ON chu.patients TO nom;        -- par colonne
REVOKE SELECT ON chu.patients FROM nom;

CREATE ROLE r_metier [IDENTIFIED BY "mdp"];
GRANT SELECT ON chu.v_accueil TO r_metier;                     -- privilège -> rôle
GRANT r_base TO r_metier;                                       -- rôle -> rôle (héritage)
GRANT r_metier TO nom [WITH ADMIN OPTION];                      -- rôle -> utilisateur
ALTER USER nom DEFAULT ROLE ALL EXCEPT r_metier;
SET ROLE r_metier [IDENTIFIED BY "mdp"];

-- Vues de contrôle
SELECT * FROM session_privs;  SELECT * FROM session_roles;
SELECT grantee, granted_role, admin_option FROM dba_role_privs;
SELECT grantee, table_name, privilege FROM dba_tab_privs WHERE owner = 'CHU';
SELECT profile, resource_name, limit FROM dba_profiles WHERE profile = 'P';
```

3. UTILISATEURS ET PROFILS

3.1. QUATRE PROFILS

Le profil est le règlement intérieur d'un compte. Au CHU, les besoins diffèrent selon les populations : aux urgences, un médecin peut ouvrir plusieurs sessions depuis des postes partagés ; une secrétaire travaille sur un seul poste ; la direction et la DPO manipulent des comptes à risque ; l'application ne doit jamais être bloquée par une expiration.

Les profils demandés (fonction standard : ORA12C_VERIFY_FUNCTION ; forte : ORA12C_STRONG_VERIFY_FUNCTION)

Paramètre (P_...)	SOIGNANT	ADMINIS-TRATIF	SENSIBLE	APPLI
SESSIONS_PER_USER	3	1	1	30
IDLE_TIME (min)	30	15	10	UNLIMITED
FAILED_LOGIN_ATTEMPTS	5	3	3	10
PASSWORD_LOCK_TIME	15 min	30 min	1 jour	1 heure
PASSWORD_LIFE_TIME	90 j	60 j	60 j	UNLIMITED
PASSWORD_GRACE_TIME	7 j	5 j	5 j	—
REUSE_TIME / REUSE_MAX	365 / 10	365 / 10	365 / 12	—
VERIFY_FUNCTION	standard	standard	forte	forte

Exercice 3.1 Créer les profils

1. [SYS] Vérifiez RESOURCE_LIMIT et l'existence des fonctions ORA12C_VERIFY_FUNCTION et ORA12C_STRONG_VERIFY_FUNCTION (DBA_OBJECTS). Si elles manquent, exécutez @?/rdbms/admin/utlpwdmg.sql puis restaurez le profil DEFAULT.
2. Créez les quatre profils du tableau.
3. Affichez leurs paramètres de mots de passe depuis DBA_PROFILES.
4. Justifiez IDLE_TIME UNLIMITED et PASSWORD_LIFE_TIME UNLIMITED pour P_APPLI. Quelle contrepartie impose-t-on ?

3.2. LES COMPTES

Comptes à créer (mots de passe imposés pour le script d'incidents)

Compte	Mot de passe	Profil
DR_ALAMI	Urgences#2026a	P_SOIGNANT
DR_BENALI	Cardio#2026b	P_SOIGNANT
DR_FASSI	Cardio#2026f	P_SOIGNANT
INF_GHAZI	Soins#2026g	P_SOIGNANT
SEC_KETTANI / SEC_LAHLOU	Accueil#2026k / Accueil#2026l	P_ADMINISTRATIF
PHA_MANSOURI	Pharma#2026m	P_ADMINISTRATIF
CPT_OUAZZANI	Compta#2026o	P_ADMINISTRATIF
DIR_RAMI	Direction##CHU26	P_SENSIBLE
DPO_SBAI	Protection##DPO26	P_SENSIBLE
APP_CHU	ChuApp##Prod2026	P_APPLI
DEV_CHU	Deploi#2026d	P_ADMINISTRATIF

Exercice 3.2 Créer les comptes

1. [SYS] Créez les douze comptes avec leur profil, le tablespace par défaut `USERS` et **aucun quota** (aucun d'eux ne crée d'objets).
2. Tentez de créer `DIR_RAMI` avec le mot de passe `Direction2026`. Quel message obtenez-vous et pourquoi ?
3. Créez le schéma `AUDIT_CHU` (sans authentification, 200 Mo sur `USERS`) : il recevra les journaux d'audit applicatifs.
4. Autorisez `DEV_CHU` à se connecter à travers `CHU` (proxy). De quel privilège `CHU` a-t-il besoin ?
5. Affichez `USERNAME`, `PROFILE`, `ACCOUNT_STATUS` et `AUTHENTICATION_TYPE` des comptes créés.

3.3. TESTER LES PROFILS



Les profils tracent les berges : chacun circule librement, mais dans des limites

Exercice 3.3 Profils en action

Pour ce test seulement, accordez directement `CREATE SESSION` à `SEC_LAHLOU`, `DR_FASSI` et `CPT_OUAZZANI` (il sera retiré au chapitre 4).

1. Ouvrez deux sessions `SEC_LAHLOU`. Message à la seconde ? Et pour `DR_FASSI` avec quatre sessions ?
2. Trois connexions `SEC_LAHLOU` avec un mauvais mot de passe, puis une avec le bon. Message ? Affichez `ACCOUNT_STATUS` et `LOCK_DATE`. Déverrouillez le compte.
3. En `CPT_OUAZZANI`, changez votre mot de passe en `Finance2027x` sans la clause `REPLACE`, puis avec. En `SYS`, tentez ensuite de lui redonner `Compta#2026o`. Que se passe-t-il ? (Ce compte n'intervient pas dans le script d'incidents.)

4. PRIVILÈGES ET RÔLES

4.1. LA MATRICE RBAC

Matrice rôles × objets à implémenter (S = SELECT, I = INSERT, U = UPDATE)

Rôle	Privilèges
R_CHU_CONNEXION	CREATE SESSION
R_SECRETAIRE	connexion ; S V_ACCUEIL, SERVICES ; S, I PATIENTS ; U(telephone, ville, mutuelle) PATIENTS ; I SEJOURS
R_INFIRMIER	connexion ; S V_SOINS, PRESCRIPTIONS
R_MEDECIN	R_INFIRMIER ; S V_ACCUEIL, SEJOURS, PATIENTS ; S, I, U DOSSIERS_MEDICAUX ; S, I, U PRESCRIPTIONS
R_PHARMACIEN	connexion ; S V_ACCUEIL, PRESCRIPTIONS ; U(statut) PRESCRIPTIONS
R_COMPTABLE	connexion ; S V_ACCUEIL ; S, I, U FACTURES
R_DIRECTION	connexion ; S V_STATS_SERVICES
R_AUDIT	connexion ; AUDIT_VIEWER ; READ sur les journaux de AUDIT_CHU (chapitre 8)
R_APPLI	connexion ; S, I, U, D sur les sept tables

Délégation : DR_BENALI, chef du service de cardiologie, reçoit R_INFIRMIER **avec** ADMIN OPTION pour habiliter lui-même les infirmiers de son service. Retenez ce point : il jouera un rôle dans l'enquête.

Exercice 4.1 Analyser la matrice

1. Pourquoi les secrétaires et le comptable passent-ils par V_ACCUEIL plutôt que par la table PATIENTS ?
2. Pourquoi R_MEDECIN hérite-t-il de R_INFIRMIER ?
3. Le pharmacien modifie le statut d'une prescription avec UPDATE ... WHERE presc_id = 8. Pourquoi a-t-il aussi besoin de SELECT sur PRESCRIPTIONS (paramètre SQL92_SECURITY) ?
4. La DPO n'a aucun accès aux dossiers médicaux. Comment peut-elle quand même contrôler leur usage ?

4.2. CRÉER LES RÔLES

Exercice 4.2 Implémenter la matrice

1. [SYS] Retirez les `CREATE SESSION` accordés directement au chapitre 3.
2. Créez les neuf rôles, alimentez-les et construisez la hiérarchie.
3. Attribuez les rôles aux comptes, y compris la délégation à `DR_BENALI`.
4. Affichez l'arbre des rôles de `DR_BENALI` avec une requête hiérarchique sur `DBA_ROLE_PRIVS`.

4.3. VÉRIFIER, ET VOIR LES LIMITES

Exercice 4.3 Campagne de tests des droits

Exécutez chaque test et comparez le résultat à l'attendu.

Tests de contrôle d'accès

Compte	Action	Attendu
SEC_KETTANI	`SELECT * FROM chu.v_accueil`	60 lignes
SEC_KETTANI	`SELECT * FROM chu.dossiers_medicaux`	ORA-00942
SEC_KETTANI	`UPDATE chu.patients SET cin = 'X' WHERE patient_id = 1`	ORA-01031
INF_GHAZI	`SELECT COUNT(*) FROM chu.v_soins`	41
INF_GHAZI	`UPDATE chu.prescriptions SET statut = 'ANNULEE'`	ORA-01031
PHA_MANSOURI	`UPDATE chu.prescriptions SET statut = 'DELIVREE' WHERE presc_id = 8`	1 ligne (puis ROLLBACK)
CPT_OUAZZANI	`SELECT * FROM chu.v_soins`	ORA-00942
DIR_RAMI	`SELECT * FROM chu.v_stats_services`	6 lignes
DIR_RAMI	`SELECT * FROM chu.patients`	ORA-00942
DR_FASSI	`SELECT diagnostic FROM chu.dossiers_medicaux WHERE dossier_id = 22`	succès (!)

1. Expliquez la différence entre ORA-00942 et ORA-01031 dans ces tests.
2. Le dernier test réussit : `DR_FASSI` (cardiologie) lit le dossier **confidentiel** d'un patient VIP de pédiatrie. Est-ce une faille des privilèges ? Que manque-t-il ?

AUDIT



Éclairer ce qui se passe réellement dans la base

R. RAPPELS

AUDIT STANDARD, AUDIT UNIFIÉ, FGA ET TRIGGERS

R. DÉFINITIONS

Audit
Enregistrement et analyse des actions effectuées sur la base, pour répondre à : qui, quoi, quand, d'où, comment, avec quel résultat. Les privilèges *empêchent* ; l'audit *prouve*.

Audit trail (piste d'audit)
Ensemble des enregistrements produits : `SYS.AUD$` pour l'audit standard, schéma `AUDSYS` (vue `UNIFIED_AUDIT_TRAIL`) pour l'audit unifié, fichiers `.aud` du système.

Audit obligatoire
Démarrage, arrêt et connexions avec privilèges d'administration (`SYSDBA`...) : toujours tracés, dans des fichiers (`AUDIT_FILE_DEST`). `AUDIT_SYS_OPERATIONS = TRUE` trace en plus toutes les actions de `SYS`.

Audit standard
Mécanisme historique piloté par `AUDIT_TRAIL` et les commandes `AUDIT`/`NOAUDIT` sur des instructions, des privilèges ou des objets. Déprécié en 21c, mais actif en 19c.

Mode mixte / mode pur
En 19c, par défaut, audit standard et audit unifié coexistent (mixte). En mode pur, seul l'audit unifié fonctionne (relink de l'exécutable).

Politique d'audit unifié
Objet nommé décrivant quoi tracer (`PRIVILEGES`, `ACTIONS`, `ROLES`, condition `WHEN`), activé ensuite par `AUDIT POLICY` pour tous, certains (`BY`) ou tous sauf certains (`EXCEPT`).

BY ACCESS / BY SESSION
Un enregistrement par instruction, ou un regroupement par session. `BY ACCESS` est la valeur par défaut et recommandée.

Audit fin (FGA)
Trace un accès seulement si une *condition sur les lignes* est vraie et si des *colonnes sensibles* sont lues ou modifiées (`DBMS_FGA`). Un gestionnaire (procédure) peut réagir en temps réel.

Trigger d'audit
Trigger écrit par le responsable applicatif pour enregistrer l'ancienne et la nouvelle valeur (`:OLD`, `:NEW`) ou faire respecter une règle métier.

Trigger système
Trigger sur un événement de la base : `AFTER LOGON ON DATABASE`, `BEFORE DROP ON schéma.SCHEMA`, `AFTER SERVERERROR`. Attributs : `ora_sysevent`, `ora_dict_obj_name`...

Transaction autonome
`PRAGMA AUTONOMOUS_TRANSACTION` : la procédure valide sa propre transaction ; la trace survit à un `ROLLBACK` ou à une erreur levée par le trigger.

Séparation des tâches
`AUDIT_ADMIN` gère les politiques et purge ; `AUDIT_VIEWER` lit seulement. Celui qui est surveillé ne tient pas la caméra.

R. SYNTAXES : AUDIT STANDARD

syntaxe

```
SHOW PARAMETER audit_trail                          -- NONE | OS | DB |
    DB,EXTENDED | XML
ALTER SYSTEM SET audit_trail = DB, EXTENDED SCOPE = SPFILE;   -- depuis la
    racine, puis redémarrer

AUDIT { option | privilège } [BY utilisateur] [BY {SESSION|ACCESS}] [WHENEVER
    [NOT] SUCCESSFUL];
AUDIT SESSION WHENEVER NOT SUCCESSFUL;          -- connexions échouées
AUDIT NOT EXISTS;                               -- objet inexistant ou
    invisible
AUDIT USER;  AUDIT ROLE;  AUDIT SYSTEM GRANT;    -- comptes, rôles,
    GRANT/REVOKE
AUDIT action [, ...] ON schéma.objet [BY ACCESS] [WHENEVER [NOT] SUCCESSFUL];
NOAUDIT ...;                                    -- même syntaxe, n'efface
    rien

-- Lecture
SELECT * FROM dba_audit_trail;   -- aussi : dba_audit_session,
    dba_audit_object, dba_audit_exists
SELECT * FROM dba_stmt_audit_opts;  SELECT * FROM dba_obj_audit_opts;
```

R. SYNTAXES : AUDIT UNIFIÉ ET FGA

syntaxe

```
CREATE AUDIT POLICY nom
  [PRIVILEGES priv [, ...]]
  [ACTIONS action [ON schéma.objet] [, ...]]
  [ROLES role [, ...]]
  [WHEN 'condition SQL' EVALUATE PER {STATEMENT | SESSION | INSTANCE}];
AUDIT POLICY nom [BY u1, u2 | EXCEPT u1] [WHENEVER [NOT] SUCCESSFUL];
NOAUDIT POLICY nom;   DROP AUDIT POLICY nom;     -- désactiver avant de
    supprimer
SELECT * FROM unified_audit_trail;               -- et
    audit_unified_enabled_policies

BEGIN
  DBMS_FGA.ADD_POLICY(
    object_schema   => 'CHU',      object_name   => 'TABLE',
    policy_name     => 'NOM',      audit_condition => 'colonne = ''valeur''',
    audit_column    => 'COL1,COL2',
    handler_schema  => 'SCHEMA',   handler_module  => 'PROCEDURE',
    statement_types => 'SELECT,UPDATE');
END;
/
-- DBMS_FGA.DISABLE_POLICY / ENABLE_POLICY / DROP_POLICY(schéma, objet, nom)
```

R. SYNTAXES : TRIGGERS D'AUDIT

syntaxe

```
-- Procédure de journalisation autonome
CREATE OR REPLACE PROCEDURE schema.log_evt (p_detail VARCHAR2) AS
  PRAGMA AUTONOMOUS_TRANSACTION;
BEGIN
  INSERT INTO schema.journal (detail) VALUES (p_detail);
  COMMIT;
END;
/
-- Trigger DML de valeurs
CREATE OR REPLACE TRIGGER schema.trg_nom
{BEFORE | AFTER} {INSERT | UPDATE [OF col, ...] | DELETE} ON chu.table
FOR EACH ROW
[WHEN (OLD.col <> NEW.col)]
BEGIN
  IF INSERTING THEN ... ELSIF UPDATING THEN ... ELSE ... END IF;   --
    :OLD.col, :NEW.col
  RAISE_APPLICATION_ERROR(-20001, 'message');                       --
    refuser l'ordre
END;
/
-- Triggers système
CREATE OR REPLACE TRIGGER schema.trg_logon AFTER LOGON ON DATABASE BEGIN ...
    END;
/
CREATE OR REPLACE TRIGGER schema.trg_ddl BEFORE DROP OR TRUNCATE ON
    chu.SCHEMA BEGIN ... END;
/
```

5. AUDIT STANDARD

5.1. ACTIVER L'AUDIT TRAIL

L'audit standard (ou traditionnel) repose sur le paramètre `AUDIT_TRAIL` et les commandes `AUDIT` / `NOAUDIT`. Il est toujours disponible en 19c (mode mixte) et reste très répandu en production.

> `AUDIT_TRAIL` est un paramètre de l'instance : il se lit dans la PDB, mais se modifie depuis la racine, avec `SCOPE = SPFILE` et un redémarrage. Les commandes `AUDIT`, elles, s'exécutent dans la PDB.

Exercice 5.1 État de l'audit standard

1. [SYS] Affichez `AUDIT_TRAIL`, `AUDIT_SYS_OPERATIONS` et `AUDIT_FILE_DEST`. Si `AUDIT_TRAIL` vaut `NONE`, passez-le à `DB, EXTENDED` et redémarrez.
2. Sur le serveur, listez les derniers fichiers du répertoire `AUDIT_FILE_DEST`. Que contiennent-ils ? Pourquoi existent-ils même si `AUDIT_TRAIL = NONE` ?

5.2. CONFIGURER L'AUDIT

Exercice 5.2 Commandes AUDIT

[SYS] Dans la PDB, configurez l'audit standard suivant :

1. les connexions **échouées** ;
2. les ordres qui échouent parce que l'objet n'existe pas (ou est invisible) ;
3. toutes les lectures, modifications et suppressions des dossiers médicaux, `BY ACCESS` ;
4. les suppressions de patients, et les `UPDATE` de factures qui **échouent** ;
5. la gestion des utilisateurs, des rôles, et les `GRANT`/`REVOKE` de privilèges système et de rôles ;
6. tout accès à la table `SYS.AUD$` elle-même.

Vérifiez ensuite la configuration dans `DBA_STMT_AUDIT_OPTS` et `DBA_OBJ_AUDIT_OPTS`. Que signifie `A/A` ? et `-/A` ?

5.3. LIRE LA TRACE

Exercice 5.3 Premiers enregistrements

1. **[DR_FASSI]** Lisez le diagnostic du dossier 22. **[SEC_KETTANI]** Tentez de lire `chu.dossiers_medicaux`. Faites une connexion `SEC_KETTANI` avec un mauvais mot de passe (une seule, pour ne pas verrouiller le compte).
2. **[SYS]** Retrouvez chaque événement : dans `DBA_AUDIT_OBJECT`, `DBA_AUDIT_EXISTS`, `DBA_AUDIT_SESSION`. Affichez l'heure, l'utilisateur, le poste, l'action, le code retour et, pour le premier, le texte SQL.
3. Que manque-t-il pour savoir *quelles* données DR_FASSI a vues, et si le patient était sensible ?

6. AUDIT UNIFIÉ

6.1. POLITIQUES DU CHU

Exercice 6.1 Mode d'audit et politiques actives

1. [**SYS**] Quel est le mode d'audit (`V$OPTION`, paramètre `Unified Auditing`) ?
2. Listez les politiques actives (`AUDIT_UNIFIED_ENABLED_POLICIES`). Lesquelles sont actives par défaut ?

Exercice 6.2 Créer et activer les politiques

[**SYS**] Créez et activez :

1. `POL_CHU_DOSSIERS` : toute lecture et modification des dossiers médicaux, et toute lecture de `V_SOINS`, pour tous les comptes **sauf** `APP_CHU` ;
2. `POL_CHU_ADHOC` : les `INSERT`, `UPDATE`, `DELETE` sur `PATIENTS`, `PRESCRIPTIONS` et `FACTURES`, **uniquement** quand le programme client est SQL*Plus ou SQL Developer (évaluation par session) ;
3. `POL_CHU_GOUVERNANCE` : création, modification et suppression de triggers, `DROP TABLE`, `TRUNCATE TABLE`, et l'usage des privilèges `SELECT ANY TABLE` et `ALTER ANY TRIGGER` ;
4. la politique prédéfinie `ORA_ACCOUNT_MGMT`.

Vérifiez dans `AUDIT_UNIFIED_POLICIES` le contenu de `POL_CHU_ADHOC`. Pourquoi exclure `APP_CHU` de la première politique ?

6.2. LA DPO, AUDITRICE

Exercice 6.3 Séparation des tâches

1. [**SYS**] Donnez au rôle `R_AUDIT` la lecture de `DBA_AUDIT_TRAIL` et de `DBA_FGA_AUDIT_TRAIL`.
2. [**DPO_SBAI**] Lisez les 10 derniers enregistrements de `UNIFIED_AUDIT_TRAIL`. Tentez ensuite `NOAUDIT POLICY pol_chu_dossiers;` puis `SELECT * FROM chu.dossiers_medicaux;`. Résultats ?
3. En quoi cette configuration respecte-t-elle l'exigence E10 ?

7. AUDIT FIN (FGA)

7.1. SIGNALER L'ACCÈS AUX DOSSIERS CONFIDENTIELS

L'audit unifié trace toute lecture des dossiers. L'exigence E5 est plus fine : seules les lectures de dossiers **confidentiels** doivent être tracées et **signalées en temps réel**. C'est le rôle de l'audit fin (package `DBMS_FGA`) avec un gestionnaire d'alerte.

Exercice 7.1 Politique FGA avec alerte

1. [**SYS**] Créez la table `AUDIT_CHU.ALERTES` (identifiant, date, utilisateur, poste, programme, objet, politique) et la procédure `AUDIT_CHU.ALERTE_FGA(p_schema, p_objet, p_politique)` qui y insère une ligne. Pourquoi doit-elle être en transaction autonome ?
2. Créez la politique `FGA_CHU_CONFIDENTIEL` sur `CHU.DOSSIERS_MEDICAUX` : condition `confidentiel = 'O'`, colonnes `DIAGNOSTIC`, `ANTECEDENTS`, `TRAITEMENT`, ordres `SELECT` et `UPDATE`, gestionnaire `ALERTE_FGA`.
3. Accordez au rôle `R_AUDIT` la lecture de la table des alertes.

Exercice 7.2 Ce qui déclenche l'alerte, ou non

[**DR_FASSI**] Exécutez les requêtes suivantes, puis [**DPO_SBAI**] consultez les alertes. Lesquelles ont déclenché une alerte ? Pourquoi ?

DR_FASSI

```
SELECT diagnostic FROM chu.dossiers_medicaux WHERE dossier_id = 5;     -- (a)
SELECT diagnostic FROM chu.dossiers_medicaux WHERE dossier_id = 22;    -- (b)
SELECT COUNT(*) FROM chu.dossiers_medicaux WHERE confidentiel = 'O';   -- (c)
SELECT dossier_id, maj_le FROM chu.dossiers_medicaux;                  -- (d)
SELECT * FROM chu.dossiers_medicaux;
-- (e)
```

8. TRIGGERS D'AUDIT

8.1. LES JOURNAUX DU RESPONSABLE APPLICATIF

Les mécanismes d'Oracle disent *qui* a agi. Le responsable de l'application veut aussi savoir *ce qui a changé* (E6), faire respecter une règle métier (E7), surveiller les connexions du compte applicatif et protéger la structure (E9). Il écrit pour cela des triggers dans le schéma `AUDIT_CHU`, qu'aucun utilisateur métier ne peut modifier.

Exercice 8.1 Les tables de journal

[**SYS**] Dans `AUDIT_CHU`, créez :

1. `JOURNAL_DOSSIERS` : date, utilisateur, utilisateur proxy, poste, programme, dossier, ancien/nouveau diagnostic, ancien/nouveau traitement, ancien/nouvel indicateur de confidentialité ;
2. `JOURNAL_EVENEMENTS` : date, type d'événement, utilisateur, proxy, poste, programme, objet, détail ;
3. la procédure `LOG_EVT(p_type, p_objet, p_detail)` en transaction autonome.

Accordez à `AUDIT_CHU` la lecture de `CHU.PERSONNEL` (directement : pourquoi pas via un rôle ?) et à `R_AUDIT` la lecture des deux journaux.

8.2. TRACER LES VALEURS

Exercice 8.2 Trigger de valeurs sur les dossiers

1. [**SYS**] Créez `AUDIT_CHU.TRG_AUD_DOSSIERS`, déclenché après toute mise à jour du diagnostic, du traitement ou de la confidentialité, pour chaque ligne, qui alimente `JOURNAL_DOSSIERS`.
2. [**DR_FASSI**] Modifiez le traitement du dossier 5 (son patient) en `'IEC + bêtabloquant'`, puis `ROLLBACK`. Puis refaites la modification et `COMMIT`.
3. [**DPO_SBAI**] Combien de lignes contient le journal ? Expliquez l'effet du `ROLLBACK`.

8.3. FAIRE RESPECTER UNE RÈGLE MÉTIER

Exercice 8.3 Seul le prescripteur modifie sa prescription (E7)

1. [**SYS**] Créez `AUDIT_CHU.TRG_CTRL_PRESC`, déclenché avant `UPDATE` ou `DELETE` de chaque ligne de `PRESCRIPTIONS` : si l'utilisateur est un médecin

(PERSONNEL.login = SESSION_USER et fonction MEDECIN) et n'est pas le prescripteur, journaliser un événement REFUS_PRESC puis lever l'erreur -20010.

2. **[DR_ALAMI]** Modifiez la dose de la prescription 8 (la sienne) puis ROLLBACK. **[DR_FASSI]** Tentez la même modification.
3. **[PHA_MANSOURI]** Marquez la prescription 8 délivrée, puis ROLLBACK. Le trigger bloque-t-il le pharmacien ?
4. Pourquoi le refus est-il encore visible dans le journal, alors que l'erreur annule l'instruction ?

8.4. CONNEXIONS ET STRUCTURE

Exercice 8.4 Trigger de connexion

1. **[SYS]** Créez AUDIT_CHU.TRG_LOGON (AFTER LOGON ON DATABASE) qui journalise (LOGON) la connexion des comptes du personnel (présents dans CHU.PERSONNEL) et de APP_CHU ; et qui **refuse** la connexion de APP_CHU si le programme client commence par sqlplus ou SQL Developer (journaliser REFUS_LOGON, erreur -20020).
2. Testez avec DR_ALAMI, puis avec APP_CHU depuis SQL*Plus.
3. Pourquoi faut-il tester ce trigger avec une session SYS ouverte à côté ?

Exercice 8.5 Protéger la structure (E9)

1. **[SYS]** Créez AUDIT_CHU.TRG_DDL_CHU, déclenché BEFORE DROP OR TRUNCATE ON chu.SCHEMA : journaliser REFUS_DDL avec l'objet et l'événement, puis lever l'erreur -20030.
2. **[DEV_CHU via proxy]** Connectez-vous avec dev_chu[chu]. Créez un index sur prescriptions(sejour_id), puis tentez TRUNCATE TABLE prescriptions.
3. Dans le journal, quelle colonne révèle la personne réelle derrière le compte CHU ?
4. DEV_CHU peut-il désactiver le trigger (ALTER TRIGGER audit_chu.trg_ddl_chu DISABLE) ? Et un DBA ? Comment le saurait-on ?

ENQUÊTE



Reconstituer les faits à partir des traces

R. RAPPELS

MÉTHODE D'ENQUÊTE ET EXPLOITATION DES TRACES

R. DÉFINITIONS

Imputabilité
Capacité d'attribuer chaque action à une personne identifiée. Elle suppose un compte par personne et, pour les comptes techniques, la colonne proxy.

Les six questions
Qui (compte, personne réelle) ; quoi (action, objet, ordre SQL) ; quand (horodatage) ; d'où (poste, IP, programme) ; comment (privilège ou rôle utilisé) ; avec quel résultat (code retour, 0 = succès).

Corrélation
Une conclusion solide croise plusieurs sources indépendantes : audit standard, audit unifié, FGA, journaux de triggers, état des comptes.

Préventif / détectif
Un contrôle préventif empêche (privilège, profil, trigger qui refuse) ; un contrôle détectif révèle (audit). Une enquête distingue ce qui a été *empêché* de ce qui a été *révélé*.

Intégrité de la trace
La trace unifiée est en lecture seule et sa purge est elle-même tracée ; `SYS.AUD$` se protège en auditant ses accès ; les journaux applicatifs se protègent par un schéma dédié sans mot de passe.

Conservation
Durée pendant laquelle on garde les traces en ligne puis archivées, fixée par la politique de l'établissement et la loi ; la purge se fait après archivage.

R. OÙ CHERCHER CHAQUE RÉPONSE

Question d'enquête et source de preuve

Question	Source
Échecs de connexion	`dba_audit_session`, `unified_audit_trail (LOGON)`, `dba_users`
Accès refusé à un objet	`dba_audit_exists`, `return_code` de `unified_audit_trail`
Lecture de données sensibles	`unified_audit_trail`, alertes FGA, `dba_fga_audit_trail`
Valeurs modifiées	journal du trigger (ancien / nouveau)
Habilitations accordées	`ORA_ACCOUNT_MGMT`, `dba_audit_trail (SYSTEM GRANT)`
Personne réelle derrière un compte	`dbproxy_username`, colonne `PROXY` des journaux

R. SYNTAXES : REQUÊTES D'ENQUÊTE ET GESTION DE LA TRACE

syntaxe

```
-- Trace unifiée : le squelette d'une requête d'enquête
SELECT event_timestamp, dbusername, dbproxy_username, os_username, userhost,
       client_program_name, action_name, object_schema, object_name,
       return_code, unified_audit_policies, fga_policy_name, sql_text
FROM   unified_audit_trail
WHERE  event_timestamp > SYSTIMESTAMP - INTERVAL '1' DAY
ORDER  BY event_timestamp;

-- Forcer l'écriture des enregistrements en attente
EXEC DBMS_AUDIT_MGMT.FLUSH_UNIFIED_AUDIT_TRAIL

-- Archiver puis purger (rôle AUDIT_ADMIN)
BEGIN
  DBMS_AUDIT_MGMT.SET_LAST_ARCHIVE_TIMESTAMP(
    audit_trail_type  => DBMS_AUDIT_MGMT.AUDIT_TRAIL_UNIFIED,
    last_archive_time => SYSTIMESTAMP - INTERVAL '90' DAY);
  DBMS_AUDIT_MGMT.CLEAN_AUDIT_TRAIL(
    audit_trail_type        => DBMS_AUDIT_MGMT.AUDIT_TRAIL_UNIFIED,
    use_last_arch_timestamp => TRUE);
END;
/
-- Trace standard : DBMS_AUDIT_MGMT.AUDIT_TRAIL_AUD_STD ; trace FGA :
    AUDIT_TRAIL_FGA_STD
```

9. LES INCIDENTS DE LA NUIT

9.1. LE SIGNALEMENT

Message de la DPO, 7 h 30

« Ce matin, la secrétaire Mme Kettani ne peut plus se connecter. Le Dr Fassi signale qu'un traitement d'un de ses patients a été modifié sans son accord. Le pharmacien a reçu une alerte sur une prescription. Et la presse locale semble savoir qu'une personnalité est hospitalisée chez nous. Pouvez-vous reconstituer ce qui s'est passé cette nuit dans la base ? »

L'enseignant exécute le script `03_incidents.sql`, qui simule plusieurs comptes au travail. Vous ne connaissez pas son contenu : seules vos traces vous permettront de répondre. Travaillez de préférence avec le compte `DPO_SBAI`, et en `SYS` seulement pour les vues que la DPO ne lit pas.

9.2. L'ENQUÊTE

Exercice 9.1 Les connexions

1. Quel compte a subi des échecs de connexion ? Combien, depuis quel poste, avec quel code d'erreur ? Dans quel état est-il maintenant ?
2. Retrouvez ces événements dans **trois** sources : `DBA_AUDIT_SESSION`, `UNIFIED_AUDIT_TRAIL` (`ORA_LOGON_FAILURES`) et l'état du compte dans `DBA_USERS`.
3. Le compte applicatif a-t-il été utilisé ? Depuis quel programme ? La connexion a-t-elle abouti ?

Exercice 9.2 Les accès aux données médicales

1. Un compte non médecin a-t-il tenté de lire les dossiers ? Avec quel résultat ? (`DBA_AUDIT_EXISTS`, `UNIFIED_AUDIT_TRAIL`)
2. Quelles alertes de confidentialité ont été levées ? Par qui, et sur quels patients ? (`AUDIT_CHU.ALERTES`, trace FGA)
3. Une secrétaire a-t-elle lu des données de soins ? Par quel chemin, et comment a-t-elle obtenu ce droit ?

Exercice 9.3 Les modifications

1. Quel dossier a été modifié cette nuit ? Par qui ? Quelles étaient les valeurs avant et après ? Le médecin auteur est-il le médecin du séjour ?
2. Quelle prescription a fait l'objet d'une tentative de modification refusée ? Par qui ?
3. Quelle opération de structure a été tentée, sous quel compte, et par quelle personne réelle ?

9.3. LE RAPPORT

Exercice 9.4 Rédiger le rapport d'enquête

Pour chacun des huit incidents, rédigez une fiche qui répond aux six questions de l'audit : **qui** (compte et personne réelle), **quoi**, **quand**, **d'où**, **comment** (privilège ou rôle utilisé), **avec quel résultat**. Indiquez la ou les sources de preuve, puis proposez des mesures correctives et préventives. Terminez par un tableau : pour chaque incident, le mécanisme qui l'a *empêché* (s'il y en a un) et celui qui l'a *révélé*.

10. SYNTHÈSE

10.1. QUESTIONS DE SYNTHÈSE

Exercice 10.1 À traiter dans le rapport

1. Pourquoi les privilèges, les rôles et les profils n'ont-ils empêché ni l'incident 3 ni l'incident 6 ? Qu'auraient-ils dû empêcher pour être « parfaits », et pourquoi est-ce impossible à l'hôpital ?
2. Comparez, sur le cas du dossier 5, ce qu'apportent l'audit standard, l'audit unifié et le trigger de valeurs.
3. Pourquoi les journaux sont-ils dans le schéma `AUDIT_CHU` et non dans `CHU` ? Qui pourrait encore les modifier, et comment le saurait-on ?
4. Le Dr Fassi soutient qu'il consultait les dossiers VIP pour un avis cardiologique. Comment l'établissement pourrait-il, à l'avenir, distinguer un accès justifié d'un accès de curiosité ?
5. Proposez une durée de conservation des traces et une procédure d'archivage et de purge (`DBMS_AUDIT_MGMT`).

10.2. BARÈME INDICATIF

Barème sur 20

Partie	Critère	Points
Ch. 2 Jeu de données	chargé et contrôlé	**1**
Ch. 3 Utilisateurs et profils	4 profils, 12 comptes, proxy	**2**
Ch. 4 Privilèges et rôles	matrice, 9 rôles, tests commentés	**3**
Ch. 5 Audit standard	configuration et lecture	**2**
Ch. 6 Audit unifié	4 politiques, DPO	**2,5**
Ch. 7 Audit fin	politique, gestionnaire, analyse (a)–(e)	**1,5**
Ch. 8 Triggers	4 triggers opérationnels	**3**
Ch. 9 Enquête	8 fiches, preuves, mesures	**4**
Ch. 10 Synthèse	qualité de l'argumentation	**1**

ANNEXES



La sécurité des données de santé est l'affaire de toute l'équipe

11. AIDE-MÉMOIRE

11.1. VUES UTILES

Vues du dictionnaire utilisées dans le TP

Vue	Contenu
DBA_USERS, DBA_PROFILES	comptes, états, profils
DBA_ROLE_PRIVS, DBA_TAB_PRIVS, DBA_COL_PRIVS	rôles et privilèges accordés
DBA_STMT / PRIV / OBJ_AUDIT_OPTS	configuration de l'audit standard
DBA_AUDIT_TRAIL, _SESSION, _OBJECT, _EXISTS	trace de l'audit standard (SYS.AUD$)
AUDIT_UNIFIED_POLICIES, _ENABLED_POLICIES	politiques unifiées
UNIFIED_AUDIT_TRAIL	trace unifiée (et FGA en audit pur)
DBA_AUDIT_POLICIES, DBA_FGA_AUDIT_TRAIL	politiques et trace FGA
AUDIT_CHU.JOURNAL_DOSSIERS / _EVENEMENTS / ALERTES	journaux applicatifs du TP

11.2. CODES D'ERREUR RENCONTRÉS

Codes d'erreur

Code	Signification
ORA-00942	objet inexistant ou invisible (aucun privilège)
ORA-01017	mot de passe invalide
ORA-01031	privilège insuffisant pour l'action
ORA-02391	trop de sessions simultanées (profil)
ORA-28000	compte verrouillé
ORA-28003 / 28007 / 28221	complexité / réutilisation / REPLACE manquant
ORA-20010 / 20020 / 20030	refus des triggers : prescription / connexion / DDL

11.3. FICHIERS FOURNIS

`01_schema_dataset.sql` : schéma `CHU` et jeu de données fictif (7 tables, 3 vues, 366 lignes).
`03_incidents.sql` : script d'incidents (enseignant).
`99_nettoyage.sql` : suppression de tout ce que le TP a créé.

99_nettoyage.sql — extrait

```
NOAUDIT POLICY pol_chu_dossiers;    DROP AUDIT POLICY pol_chu_dossiers;
NOAUDIT POLICY pol_chu_adhoc;       DROP AUDIT POLICY pol_chu_adhoc;
NOAUDIT POLICY pol_chu_gouvernance; DROP AUDIT POLICY pol_chu_gouvernance;
EXEC DBMS_FGA.DROP_POLICY('CHU','DOSSIERS_MEDICAUX','FGA_CHU_CONFIDENTIEL')
DROP TRIGGER audit_chu.trg_logon;
DROP USER audit_chu CASCADE;
DROP USER chu CASCADE;
```

Liste des figures

1 Le modèle du schéma CHU 10
2 Les profils tracent les berges : chacun circule librement, mais dans des limites 18

Liste des tableaux

1 Les acteurs du CHU et leurs comptes Oracle 7
2 Déroulement conseillé (durée totale : 8 h) 9
3 Les tables et les vues sensibles 10
4 Les profils demandés (fonction standard : ORA12C_VERIFY_FUNCTION ; forte : ORA12C_STRONG_VERIFY_FUNCTION) 16
5 Comptes à créer (mots de passe imposés pour le script d'incidents) 17
6 Matrice rôles × objets à implémenter (S = SELECT, I = INSERT, U = UPDATE) 19
7 Tests de contrôle d'accès 20
8 Question d'enquête et source de preuve 34
9 Barème sur 20 37
10 Vues du dictionnaire utilisées dans le TP 39
11 Codes d'erreur 39